

Eight Steps in a Repository

A person still has to approve every change. These eight steps are meant to make that approval mean something.

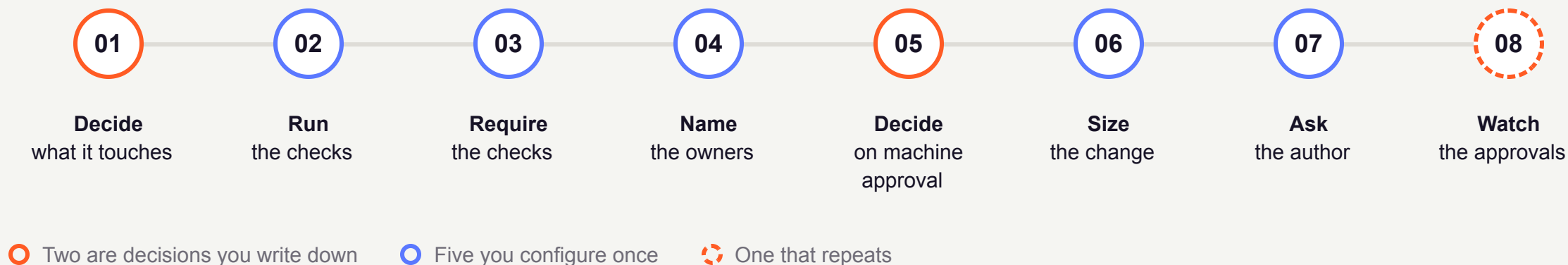
Seven of them are one file you add, or one setting you turn on. The eighth is half an hour of reading, once a month.



THE AGENDA

Eight steps, in the order they get done

When a change is approved, the approval should mean a person read it and understood it. These steps are what makes that true inside one repository.



Everything on this track is a file in the repository or a setting on it, except the last, which is half an hour a month. **Nothing here needs a platform team, a budget, or a vendor.**

Decide what this repository touches

The answer decides how strict the later steps should be. A repository that moves money needs more care than one that runs an internal dashboard. If nobody decides, both are treated the same.

- Does it move money, hold customer data, or neither?
- Write the answer in the README, under “Review rules”, with the date
- Say what follows: reviewers, machine approval, security review
- Revisit it when the repository starts touching something new

```
## Review rules
```

```
Touches: customer money, through  
settlement and the ledger. 2026-09-28
```

```
What follows: two reviewers on every  
change, security review required,  
no auto-merge.
```

Financial rules ask for the same thing. They say the policy must be **based on a risk assessment**, and they never say the assessment has to be long. One dated sentence in the README is enough.

Make the mechanical checks run on every change

Some things a machine can check on its own, with no judgement needed.

Those checks should run before a person reads the change, so the reviewer never spends attention on them.

- Formatting and style, so no reviewer spends time on spacing
- The test suite
- Static analysis, a tool that reads the code and flags known mistakes
- Dependency scanning, a warning when a library has a known problem
- Secret scanning, which catches a password or key committed by accident

The first three run from one file, `.github/workflows/`. Dependency scanning is `.github/dependabot.yml`. Secret scanning is a switch in the repository settings. Google's reason for all of it: leave human review for **the questions a machine cannot answer**.

Make the checks from step 2 block a merge

By default a failing check does not stop anyone. Running a check and requiring it are two different settings. A check that runs is a suggestion. A check that blocks a merge is a rule.

- Name each check in the list, one by one. A check you add later is not required until you add it here too
- Require a pull request before anything is merged
- Stop anyone pushing straight to the main branch
- Cancel an approval when new commits arrive after it

```
Settings > Branches > main
```

```
[x] Require a pull request before merging
```

```
[x] Dismiss stale approvals when new  
    commits are pushed
```

```
[x] Require status checks to pass
```

```
    lint
```

```
    test
```

```
    static-analysis
```

```
[x] Restrict who can push to this branch
```

The last one matters most. Without it, someone can approve a change and then add code nobody has read.

Name the owners of the paths that carry risk

Step 1 described the whole repository. Inside it, some folders matter more than others. A file named CODEOWNERS says who has to review which folders.

- List the folders that move money or hold customer data
- Give each one a named person or team
- Leave the rest of the repository as it is
- Switch on “require review from code owners” in the settings. Without that, the file only suggests reviewers

```
# .github/CODEOWNERS
# Everything, unless a later line overrides it.
*                               @payments-team

# The paths that move money.
/src/settlement/                @settlement-owners @security
/src/ledger/                    @settlement-owners

# Anything that changes how the service is deployed.
.github/workflows/              @platform
```

Order matters. **The last line that matches a file wins**, so the line covering everything goes first, and the folders that carry risk go below it.

Decide whether a machine may approve here

Since 1 September 2026, a review from GitHub Copilot can count as the approval a repository requires. It stays off until somebody turns it on. The answer follows from step 1: what the repository touches decides it.

- Take the answer from the tier in step 1, not change by change
- Decide whether a Copilot review counts toward this repository's required approvals
- Write the answer and the reason, not the answer alone
- Revisit it when what the repository touches changes

```
## Review rules
```

```
Touches: customer money, through  
settlement and the ledger. 2026-09-28
```

```
Machine approval: not allowed here. A  
machine approval would be the only  
approval on a change that moves money,  
and nothing logs that it happened.  
2026-09-28
```

A repository that moves money and one that runs an internal dashboard may answer differently. Both answers belong under the same **“Review rules”** heading, because the setting has no room for a reason and the organisation-wide log records nothing when a machine approves.

Keep the change small enough to be read

Steps 3 and 4 hand the change to a reviewer. They assume that person can actually read it. A change too large to read in one sitting still gets approved.

- Agree a size this repository does not go past without saying why
- Check it in CI: a warning everywhere, a failure where step 1 said the repository carries risk
- Keep the pull request marked as a draft until the checks pass, so nobody is asked to read a broken change
- Split what is too large, or say in the description why it could not be split

Beyond **200 lines an hour** reviewers found about half the defects. One in four AI-written pull requests runs past 400 lines. Adyen keeps its automated changes under five files, which is what makes them “**fast to review, easy to approve**”.

Ask the author for what no check can see

A template fills every new pull request with the same questions, so the author answers them before anyone is asked to review. The second question is the one that matters.

- What changed, and why it belongs here
- What you want looked at, specifically
- How to undo it
- Which parts an agent wrote

```
<!-- .github/pull_request_template.md -->

## What this changes, and why it belongs here

## What I want looked at
<!-- Name one thing. "Anything" is not
      an answer. -->

## How to undo it

## What an agent wrote, and what I checked
```

Across **80.000 pull requests**, a description rarely changed the review. The exception explained the code *and* asked what to look at. A summary alone is not enough.

Watch for the day the approvals stop meaning anything

Steps 1 to 7 can all be in place and still stop working, and nothing announces it. The checks stay green, the approvals keep arriving, the template keeps getting filled in, and none of that requires anyone to have read the change.

Once a month, read ten merged pull requests and ask what you are seeing:

- Approvals that arrived a minute after opening, on changes too long to read in a minute
- Template sections filled with “N/A”, the branch name, or the commit message pasted back
- The same person approving everything, including the parts they do not work on
- Code owners approving paths they have never otherwise touched

No setting reports any of this. **Somebody has to go and look.**

WHAT THE LIST CANNOT DO

**It makes the decision cheaper to reach.
It does not make the decision.**

Steps 2 to 4 clear away everything a machine can check. Steps 6 and 7 give the reviewer a change they can read, and an explanation of it. Step 8 checks that someone is still reading. None of them decides whether the change should exist. A person does that.

And I have not run this list against our own repositories. I do not know how many of these settings are already on, which makes step 3 the first thing I owe the argument.